Control-Tree Compilation

Two-qubit gate reduction for quantum circuits with shared multi-controlled logic

[Author name]

`ctc-optimizer` 0.1.0

September 2026

Abstract

Many quantum algorithms are written as *quantum if-statements*. A register of selector qubits (a direction, a copy, a stage of an algorithm) guards a small reversible program: an increment, a register swap, a phase or a rotation. Compilers synthesise each multi-controlled gate of such a program on its own, so the conjunction of the shared controls is recomputed for every gate. We present *Control-Tree Compilation* (CTC), a compiler pass for Qiskit circuits that reasons about guarded operations directly.

CTC works in two stages. Stage A applies four exact rewrites to the gate list before any synthesis:

- A1: commutation-aware cancellation;
- A2: Pauli-group collapse;
- A3: one-literal cube merging;
- A4: multiplexor detection.

Stage B lowers the result with three further steps:

- B2: an AND-cache that holds each shared conjunction on a clean ancilla, built from relative-phase Toffoli gates;
- B1: reordering of commuting guarded runs;
- B3: sibling flips, which turn a cached $K \wedge l$ into $K \wedge \neg l$ for 2 CX instead of 6.

Every step is an algebraic identity. Each is stated with a proof and a CX cost analysis.

The method ships as a Python package with a single entry point, `optimize(circuit)`. A built-in search guarantees the result is never worse than Qiskit's own optimisation of the input. On 60 benchmark circuits all 865 simulated results are exact.

On selector-controlled increment ladders, CTC reduces two-qubit gates by up to 18.1× over Qiskit at optimisation level 3. On the full timestep of a quantum magnetostatic curl-curl solver it gives 2 461→520 CX against the standard per-gate clean-ancilla technique. On random guarded circuits the median reduction is 3.48×, against 1.05× for pytket's `FullPeepholeOptimise`. The QFT, which has no multi-controlled logic, is returned unchanged. The ablations show that the AND-cache carries most of the gain whenever ancillas are available. Stage A's rewrites are large on their own, but largely subsumed once the cache is on; multiplexor detection is the exception. Run ordering never helped in these benchmarks.

Contents

1 Introduction **2**

2 Background **3**

3 The guarded-gate IR **4**

3.1 Commutation 4

4 Stage A: exact rewrites before synthesis **5**

4.1 A1 — commutation-aware cancellation 5

4.2 A2 — Pauli-group collapse 6

4.3 A3 — one-literal cube merging 6

4.4 A4 — multiplexor detection 6

5 Stage B: synthesis with shared control **6**
5.1 AND nodes and the relative-phase lemma . . . 6
5.2 The cost model . . . 7
5.3 B2 — the AND-cache . . . 7
5.4 B1 — ordering guarded runs . . . 8
5.5 B3 — sibling flips . . . 8
5.6 Ancilla budgets . . . 9

6 The `optimize` driver **9**

7 Evaluation **10**
7.1 Circuit families . . . 10
7.2 What each step contributes . . . 11
7.3 Random guarded circuits . . . 12
7.4 A quantum PDE solver . . . 13
7.5 Ancilla trade-off . . . 14
7.6 Run time . . . 14

8 Related work **14**

9 Limitations **16**

10 Conclusion **16**

A Reproducing the results **16**

1 Introduction

Two-qubit gates dominate both error and cost on current hardware. Minimising them is the first job of a compiler. For circuits built from multi-controlled gates, most of the two-qubit cost is *control logic*: the AND of the controls, computed and uncomputed around a comparatively cheap core operation. A Toffoli costs 6 CX. A c-controlled X costs $6c - 6$ CX with $c - 2$ clean ancillas [1]. Without ancillas it costs quadratically more.

A common way to write quantum algorithms concentrates this logic in a particular shape. Consider a quantum walk or lattice-Boltzmann shift operator: "if the direction register holds d, add 1 to the position register". Implemented directly, every gate of the +1 ladder carries the full condition on the direction register, plus its own register bits. The same holds for oracle-controlled blocks, controlled register permutations and multiplexed state preparation. The program is a tree of quantum if-statements, but it reaches the compiler as a flat list of independent multi-controlled gates.

Existing tools do not recover this structure:

- Qiskit's high-level synthesis runs before its optimisation passes, and synthesises each multi-controlled gate independently.
- Peephole and template optimisers [10, 15] see only local windows of the already-synthesised circuit.
- Search-based optimisers [16, 17] are general but do not target shared control.

In our measurements, Qiskit's optimisation level 3 removed no CX at all from a 74-gate block in which every gate shares a 7-qubit condition. pytket's `FullPeepholeOptimise` recovers little on such circuits (§7).

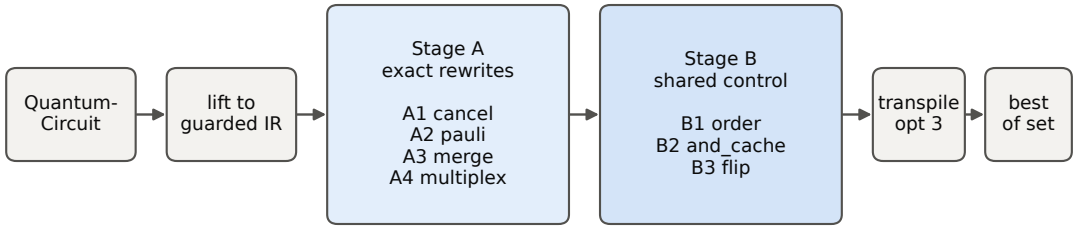


Figure 1: The CTC pipeline. Stage A rewrites the guarded-gate list exactly. Stage B lowers it to small gates with a shared AND-cache on clean ancillas. The driver keeps the best of several transpiled candidates.

Contributions. Figure 1 gives an overview.

1. **A guarded-gate IR** (§3). Every gate is represented as (control cube, core operation, targets), together with a sound commutation test built from five algebraic rules (§3.1).
2. **Four exact rewrites** on the IR (Stage A, §4): cancellation, Pauli collapse, cube merging and multiplexor detection. Each comes with a correctness proof and its CX effect.
3. **An AND-cache lowering** (Stage B, §5). It extends Maslov's relative-phase Toffoli argument from one gate to a whole gate sequence. It is driven by a look-ahead cost model and supports an ancilla budget. Two refinements go with it: run ordering, and *sibling flips*, which re-target a cached conjunction for 2 CX using a numerically derived phase correction.
4. **A one-call package** with a never-worse guarantee (§6).
5. **An evaluation** (§7) on worked examples, six circuit families, 20 random guarded circuits and a quantum PDE solver. It includes cumulative and leave-one-out ablations of every step, comparisons with per-gate clean ancillas and pytket, and ancilla trade-off curves. Every result up to 26 qubits was verified by simulation.

2 Background

Cost of multi-controlled gates. The table below gives the CX cost of a c-controlled X in the basis $\{\mathrm{CX}, R_z, \sqrt{X}, X\}$ after Qiskit 2.5 optimisation level 3, as measured for this work.

construction	ancillas	CX	$c = 7$	$c = 11$
Qiskit default, no spare qubits (Huang–Palsberg / Vale et al.)	0	superlinear	180	564
dirty v-chain (Iten et al. [2])	$c-2$ dirty	$8c-6$	50	82
clean v-chain, relative-phase Toffolis (Maslov [1])	$c-2$ clean	$6c-6$	36	60
conditionally clean (Khattar–Gidney [4])	1 clean	$6c-6$	36	60

Maslov's construction computes the AND of the first $c-1$ controls with a ladder of $c-2$ *relative-phase* Toffolis (RCCX, 3 CX each). It then applies one ordinary Toffoli onto the target (6 CX) and uncomputes the ladder: $2(c-2)\cdot 3+6 = 6c-6$. An RCCX implements the Toffoli map up to a diagonal phase that depends on the basis state. The phase is harmless because each compute rung is undone by an identical uncompute rung.

Clean and dirty ancillas. A *clean* ancilla is known to be in $|0\rangle$. A *dirty* one may hold anything and must be restored. Qiskit's transpiler, when spare qubits exist, uses them as dirty ancillas. It uses them as clean ancillas only when the flag `qubits_initially_zero` is set. That flag is unsound for circuits applied mid-computation, and we always pass `qubits_initially_zero=False`. The clean ancillas used by CTC are qubits added to the circuit, initialised to $|0\rangle$ and returned to $|0\rangle$.

Exact transpilation. At optimisation levels 2 and 3, Qiskit's `ElidePermutations` pass deletes SWAP gates and records the permutation only in the circuit's `layout.final_layout`. A QFT, which ends in SWAPs, therefore transpiles to a circuit that on its own implements a *different* operator, correct only up to a relabelling of output qubits. Throughout this work, for CTC and every baseline alike, transpilation uses the level-3 preset pass manager with `qubits_initially_zero=False` and `ElidePermutations` removed (`ctc_optimizer.transpile_exact`). Every circuit compared therefore implements the input operator on the same qubits. Counts for circuits containing SWAP gates are consequently higher than Qiskit's default output, which charges nothing for SWAPs it has moved into the layout.

Uniformly controlled rotations. A multiplexor applies $R_y(\theta_s)$ to a target, where s is the value of m select qubits. It can be synthesised with 2^m CX and 2^m single-qubit rotations [12]. The same operation written as 2^m separate m-controlled rotations costs far more.

3 The guarded-gate IR

Definition 1 (guarded gate)**.** A *cube* $C = \{(q_1, p_1), \ldots, (q_r, p_r)\}$ is a set of literals: qubit q_i with polarity $p_i \in \{0, 1\}$. Its projector is $P_C = \bigotimes_i |p_i\rangle\langle p_i|_{q_i}$. A guarded gate $g = (C, U, T)$ with core unitary U on target qubits T, disjoint from the qubits of C, is the operator

$$g \;=\; P_C \otimes U_T \;+\; (I - P_C) \otimes I_T.$$

The IR distinguishes these *core kinds*:

- X, Z, H, $R_y(\theta)$, SWAP;
- `gate1`: any other single-qubit gate with a known 2×2 matrix (S, T, R_z, P, Y, $\sqrt{X}$, ...);
- `phase`(φ): $e^{i\varphi} P_C + (I - P_C)$, which has no target;
- `ucry`: a multiplexor, produced by A4;
- `raw`: any other instruction, kept verbatim. This covers measurements, resets, classical control flow, gates with unbound parameters and custom multi-qubit gates.

Lifting. `lift` reads a `QuantumCircuit`. A `ControlledGate` whose base gate is one of the core kinds becomes a guarded gate, with cube given by its control qubits and `ctrl_state`. One subtlety: some Qiskit gates are not described by their base gate. For example, `CUGate` carries a phase its base `UGate` lacks. CTC therefore accepts a controlled gate only after checking $\mathrm{mat}(g) = P_s \otimes \mathrm{mat}(\mathrm{base}) + (I - P_s) \otimes I$ for up to 6 controls, or when its class is known to be a plain control. Otherwise the gate is `raw`. Each guarded gate also records the order in which its controls were listed (§5.4).

Reads and writes. A gate *writes* a qubit if it can change that qubit's computational-basis value. For X, H, R_y, SWAP and non-diagonal `gate1`, the targets are written. For Z, `phase` and diagonal `gate1`, nothing is written. For `raw`, all of its qubits are written. A diagonal gate is never considered to write, which is what makes rule 3 below and the AND-cache argument work.

3.1 Commutation

All reordering in CTC relies on one *sufficient* test. Write $Q(g)$ for the support of g, $W(g)$ for its written qubits and $K(g)$ for its control qubits.

Proposition 1 (commutation rules)**.** *Two guarded gates g, h commute if any of the following holds:*

1. $Q(g) \cap Q(h) = \emptyset$*;*
2. *both are diagonal;*
3. *g is diagonal and $Q(g) \cap W(h) = \emptyset$ (or symmetrically);*
4. *there is a qubit $q \in K(g) \cap K(h)$ with opposite polarities in the two cubes and $q \notin W(g) \cup W(h)$;*

Table 1: Worked examples, one per step: two-qubit gates after transpilation (ancillas in brackets). "Clean per gate" replaces each MCX with ≥ 3 controls by Maslov's construction on shared ancillas. "Without" is CTC with the step's prerequisites only. "With" adds the step. All results verified exact.

step	qubits	Qiskit opt 3	clean per gate	without	**with the step**
A1 `cancel`	6	29 (0)	19 (1)	—	**1** (0)
A2 `pauli`	4	56 (0)	52 (1)	—	**6** (0)
A3 `merge`	4	28 (0)	18 (1)	—	**6** (0)
A4 `multiplex`	3	32 (0)	—	—	**4** (0)
B2 `and_cache`	8	120 (0)	62 (3)	—	**27** (3)
B1 `order`	8	104 (0)	63 (2)	40 (6)	**33** (3)
B3 `flip`	8	104 (0)	36 (2)	25 (3)	**24** (3)

5. *$K(g) \cap T(h) = K(h) \cap T(g) = \emptyset$, and the core operators commute on $T(g) \cup T(h)$.*

A `raw` *gate commutes only by rule 1.*

Proof. *(1)* and *(2)* are immediate.

(3) If h does not change the basis values of $Q(g)$, then h is block-diagonal in the computational basis of $Q(g)$, i.e. $h = \sum_b |b\rangle\langle b| \otimes h_b$. Meanwhile g is $\sum_b e^{i\phi_b}|b\rangle\langle b| \otimes I$, so they commute.

(4) Neither gate changes q, so $\mathcal{H} = \mathcal{H}_{q=0} \oplus \mathcal{H}_{q=1}$ is invariant under both. On each summand at most one of the two gates acts non-trivially, because their cubes require opposite values of q.

(5) Both are block-diagonal in the basis of $K(g) \cup K(h)$, since neither touches the other's controls. On every block they act as U_g^a and U_h^b with $a, b \in \{0, 1\}$, and these commute by assumption. □

For rule 5, core commutation is checked on matrices of at most 4 target qubits. The test is conservative: when no rule applies, the gates are treated as non-commuting.

4 Stage A: exact rewrites before synthesis

Stage A repeats its four steps to a fixed point, in the order A2, A1, A3, A4. Running A2 first stops A1 from cancelling one X out of a $ZXZX$ group and so breaking it. Each step is local: it scans forward from a gate across commuting neighbours only. Table 1 gives a worked example per step.

4.1 A1 — commutation-aware cancellation

Rewrite. Scan forward from g over gates that commute with g (Prop. 1). If a gate h with the same cube, kind and targets is found, and it commutes with every gate in between, then:

- for self-inverse cores (X, Z, H, SWAP), delete both;
- for R_y, replace both by $R_y(\theta_g + \theta_h)$. The angle is taken mod 4π; an angle of 2π becomes a cube phase of π, because $R_y(2\pi) = -I$;
- for `gate1`, delete both when $U_h U_g = e^{i\varphi} I$, leaving a cube phase φ if $\varphi \neq 0$;
- for `phase`, add the angles.

Correctness. Moving h back next to g passes only gates it commutes with. Then $g\,h = P_C \otimes U_h U_g + (I - P_C) \otimes I$.

Cost. Removing a pair of c-controlled X gates saves $2(6c-6)$ CX with clean ancillas, or twice the ancilla-free cost without them. Qiskit's own cancellation runs after synthesis and cannot see identical *multi-controlled* gates. In the worked example (an MCX pair around a CX that shares one control) it leaves 29 CX, while A1 leaves 1.

4.2 A2 — Pauli-group collapse

Rewrite. A maximal contiguous window of X/Z gates with the same cube C and the same single target is multiplied out as $U = \omega X^a Z^b$, with $\omega \in \{\pm 1, \pm i\}$. It is replaced by at most three gates: Z if $b = 1$, X if $a = 1$, and $\texttt{phase}(\arg \omega)$ on C if $\omega \neq 1$. The rewrite is applied only when it shortens the window.

Correctness. $P_C \otimes \omega X^a Z^b + (I - P_C) \otimes I$ factors as $(\omega P_C + I - P_C) \cdot (P_C \otimes X^a Z^b + (I - P_C) \otimes I)$.

Cost. The pattern $Z\,X\,Z\,X$, which appears when a copy of a register is negated and swapped twice, gives $XZXZ = (-iY)^2 = -I$. The whole group, two c-controlled X and two c-controlled Z, becomes $\texttt{phase}(\pi)$ on C. That is an MCZ over the c cube qubits, i.e. an MCX with $c-1$ controls: $6(c-1) - 6$ CX with clean ancillas, instead of about $4(6c-6)$. Under Stage B it costs *nothing*: if the cube's conjunction is cached on an ancilla, the phase is a single Z on that ancilla. In the worked example ($c = 3$): $56 \to 6$ CX.

4.3 A3 — one-literal cube merging

Rewrite. Two gates with the same core and targets, whose cubes have the same qubits and differ in the polarity of exactly one qubit q, are merged into one gate on $C \setminus \{q\}$. The second gate must commute back to the first.

Correctness. $P_{C,q=0} + P_{C,q=1} = P_{C \setminus q}$, and the two projectors are orthogonal, so the product of the two gates is $P_{C\setminus q} \otimes U + (I - P_{C\setminus q}) \otimes I$. This is the reversible-logic counterpart of a Karnaugh merge [11].

Cost. Two c-controlled gates become one $(c-1)$-controlled gate: from $2(6c-6)$ to $6c-12$ CX for MCX with clean ancillas. Worked example ($c = 3$): $28 \to 6$ CX.

4.4 A4 — multiplexor detection

Rewrite. Collect a group of R_y gates on the same target that are adjacent modulo commutation and have non-empty cubes. Let C_0 be the literals common to all their cubes, and $S = \{s_1, \ldots, s_m\}$ the remaining control qubits ($m \leq 4$). Build the angle table

$$\theta(s) = \sum_{g:\ C_g \setminus C_0 \text{ agrees with } s} \theta_g, \qquad s \in \{0,1\}^m,$$

and replace the group by one multiplexor on S, guarded by C_0.

Correctness. R_y gates on one qubit commute and their angles add. For each value s of the select qubits, the product of the group is $R_y(\theta(s))$.

Cost. A multiplexor costs 2^m CX. The same table written as 2^m separate m-controlled R_y costs $\Theta(2^m m)$ or more. Worked example ($m = 2$, four C^2R_y): $32 \to 4$ CX. When C_0 is non-empty and Stage B supplies a single control line for it, that line is folded in as one more select qubit whose other half of the table is zero, giving 2^{m+1} CX.

5 Stage B: synthesis with shared control

Stage B turns the guarded-gate list into a circuit of small gates. The central idea is to keep conjunctions of literals alive on clean ancillas for as long as later gates can use them.

5.1 AND nodes and the relative-phase lemma

An *AND node* is a set of literals K whose conjunction is held on an ancilla a_K. A singleton node is free: it is the literal's own qubit. A node $K \cup \{l\}$ is built from a parent node K and a literal l by one RCCX onto a fresh clean ancilla (3 CX). It is removed by the same RCCX, which is self-inverse, for another 3 CX. The *control line* of a node is a_K, or the literal itself for a singleton.

Lemma 1 (relative-phase cancellation across many gates)**.** *Let V be an RCCX with controls (u, l) and target a, so that $V|u, l, 0\rangle = e^{i\phi(u,l)}|u, l, u \wedge l\rangle$. Let M be any sequence of gates that (i) does not write u or l, and (ii) touches a only as a control or through diagonal gates. Then $V^\dagger M V$ acts on $|u, l, 0\rangle \otimes |\psi\rangle$ as $|u, l, 0\rangle \otimes M_{u\wedge l}|\psi\rangle$, where M_x is M with a fixed to x. In particular the phases ϕ cancel exactly.*

Proof. Fix basis values u, l; the general case follows by linearity. After V the state is $e^{i\phi}|u, l, x\rangle|\psi\rangle$ with $x = u \wedge l$. By (i) and (ii), M maps it to $e^{i\phi}e^{i\chi(x)}|u, l, x\rangle M'_x|\psi\rangle$, where the diagonal part contributes $e^{i\chi(x)}$. Since V is unitary and $V|u, l, 0\rangle = e^{i\phi}|u, l, x\rangle$, $V^\dagger|u, l, x\rangle = e^{-i\phi}|u, l, 0\rangle$. The phases cancel, and $M_x = e^{i\chi(x)}M'_x$. □

Applied recursively, the lemma covers chains (a_K serves as the u of its children) and trees of nodes. It holds provided that, between a node's compute and its uncompute, nothing writes its literal qubits and its ancilla is only read or phased. CTC enforces exactly that invariant.

5.2 The cost model

Each gate is emitted under the control line of its deepest usable cached node, plus m further residual control literals. The CX cost used for decisions is:

core	$m = 0$	$m = 1$	$m = 2$	$m \geq 3$
X, Z	0	1 (CX/CZ)	6 (CCX)	$6m - 6$
`phase`(π)	0	0 (Z on the line)	1 (CZ)	$6(m-1) - 6$
`phase`(φ)	0	0	2 (CP)	$6m$
SWAP	3	7 (Fredkin)	$2 +$ MCX(3)	$2 + 6(m+1) - 6$
R_y, H, `gate1`	0	2	$6(m-1) + 2$	$6(m-1) + 2$
`ucry` (s selects)	2^s	2^{s+1}	$2^{s+1} + 6(m-1)$	$2^{s+1} + 6(m-1)$

A node costs 6 CX in total (compute and uncompute), and a sibling flip costs 2 (§5.5).

5.3 B2 — the AND-cache

```
for i, g in enumerate(gates):
    uncompute every node whose literal qubits g writes      # keeps Lemma 1 valid
    if g is raw: emit g; continue
    K := deepest cached node with K subset of cube(g)       # singletons are free
    repeat:
        for each literal l in cube(g) - K:
            gain(l) := sum over later gates h that can still use K+{l}
                         of cost(h, m_h(K)) - cost(h, m_h(K+{l}))
            price(l) := 0 if cached, 2 if a sibling can be flipped, else 6
        pick l maximising gain(l) - price(l); stop if negative
        K := K+{l}   (cached / flipped / built; on budget pressure, evict the leaf
                      node whose next use is farthest away)
    emit core(g) controlled by line(K) and the residual literals cube(g) - K
    uncompute leaf nodes that no later gate can use
uncompute everything left
```

"Later gates that can use K" are those whose cube contains K, up to the first gate that writes a qubit of K. An index of literal positions and write positions answers this in $O(\log G)$ per query plus the size of the answer.

Why this generalises Maslov. For a single gate with c controls and no later users, the loop extends K until the residual cost stops falling. That is exactly Maslov's ladder ($c - 2$ RCCX pairs and one Toffoli), so the per-gate cost is never above $6c - 6$. With later users, the shared prefix is paid for once.

Controlled increments become staircases. For a +1 ladder on bits $b_0, \ldots, b_{k-1}$ under a selector cube S, the gate for b_j has cube $S \cup \{b_0, \ldots, b_{j-1}\}$. The prefix nodes $S \cup \{b_0..b_{j-1}\}$ are used by every later gate that reads those bits, and they stay valid until those bits are written. The cache therefore keeps a staircase of prefix ANDs: $O(k)$ RCCX in total, instead of $\sum_j O(|S| + j) = O(k|S| + k^2)$ for per-gate synthesis. This is Gidney's temporary-AND incrementer [5], arising from the cost model with no special case.

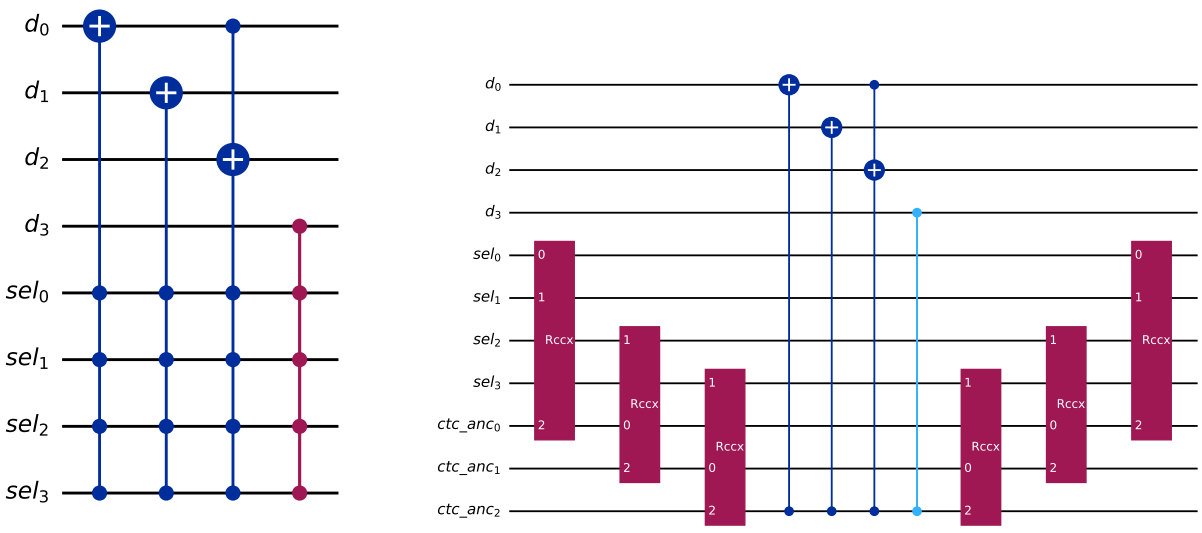


Figure 2: B2 worked example. Left: four gates guarded by the selector `sel`. Right: CTC output before transpilation. The selector AND is computed once onto `ctc_anc_2` by relative-phase Toffolis, used by four small gates, and uncomputed. Two-qubit gates after transpilation: 120 (Qiskit) → 27 (CTC).

Worked example. Four gates share a 4-qubit selector: two MCX, one MCX with an extra control, and one MCZ (Figure 2). Qiskit gives 120 CX; clean ancillas per gate give 62; the AND-cache gives 27. The selector conjunction is built once by a chain of three RCCX gates and consumed by four gates: two CX, one Toffoli (the extra control) and one CZ (the MCZ). It is then uncomputed by the mirrored chain.

5.4 B1 — ordering guarded runs

Scan the list for maximal segments whose gates all read a common set Q of at least two control qubits that no gate in the segment writes. Gates in the segment whose literals on Q differ commute, by rule 4 of Prop. 1. The segment is therefore regrouped by its value on Q, keeping the order within each group. The groups are then ordered along a minimum-Hamming-distance path, computed exactly by Held–Karp for up to 12 groups and greedily beyond. Consecutive groups then share the longest possible prefix of cached literals. Commutation is re-checked pairwise before any move.

The builder's control order matters too. Multi-controlled synthesis ANDs controls in list order, and consecutive gates sharing a leading prefix are what Qiskit can cancel. In our measurements, re-emitting a 74-gate block with its controls sorted by qubit index nearly doubled its CX count (1 176 → 2 190). CTC keeps the source circuit's control order wherever it emits multi-controlled gates.

Worked example (two selector values interleaved): without B1 the cache needs 40 CX on 6 ancillas; with B1, 33 CX on 3.

5.5 B3 — sibling flips

Consecutive runs often differ in the value of a single selector literal. A leaf node holding $K \wedge l$ can then serve the next run as $K \wedge \neg l$, since

$$(K \wedge l) \oplus K \;=\; K \wedge \neg l,$$

so a single CX from the line of K onto the ancilla re-targets it. The RCCX that computed the node and the RCCX that will uncompute its sibling are different gates, so Lemma 1 no longer cancels their phases by itself. On the states the ancilla can hold, however, the difference is a diagonal on (K, l):

$$\mathrm{RCCX}_{\neg l} \cdot \mathrm{RCCX}_{l}\,|k, l, k \wedge l\rangle \;=\; D(k,l)\,\mathrm{CX}_{k\to a}|k, l, k \wedge l\rangle.$$

CTC computes D once per polarity case from the installed Qiskit's RCCX matrix. For positive literals the phases on $(k,l) = (0,0),(0,1),(1,0),(1,1)$ are $(1, 1, -i, +i)$, i.e. $D = S_K^\dagger \cdot \mathrm{CZ}_{K,l}$. A flip therefore costs one

CX, one CZ and a phase gate: **2 CX instead of 6**. The cache prices a flip at 2 and keeps a finished leaf alive when its sibling is needed next.

A flip is valid only if the cached node was *built* as $K + \{\neg l\}$. A node holding the same literal set but built in a different order has a different last literal, so flipping it would toggle the wrong one. The package's randomised tests found this case; CTC now checks the construction record. Two further implementation errors were caught while preparing this paper, both by verification rather than by inspection:

- the reserve-an-ancilla variant was skipped at a budget of exactly one ancilla;
- Qiskit's SWAP elision (§2);
- pytket's `tk_to_qiskit` orders registers by name, which silently permutes the qubits of multi-register circuits. Its results are now mapped back by (register, index).

Both are fixed, and every number reported here was produced after the fixes.

Worked example (runs with selector values 0111 and 1111): 25 → 24 CX. Here Qiskit's optimiser already cancels part of the naive uncompute-recompute pair, so the flip adds only a small gain.

5.6 Ancilla budgets

With `max_ancillas`= B, node construction fails when B ancillas are busy. The cache then evicts the leaf node whose next use is farthest away, or falls back to residual controls. Residual MCX gates with $m \geq 3$ controls use Maslov's construction if $m - 2$ ancillas are free, and Khattar–Gidney's 1-ancilla construction [4] if one is. Only when no ancilla is free do they use Qiskit's default synthesis. Optionally one ancilla is held back from the cache so that the Khattar–Gidney fallback is always available; the driver tries both settings.

6 The `optimize` driver

```
out = optimize(circuit, max_ancillas=None, steps=ALL_STEPS,
               basis_gates=("cx","rz","sx","x"), optimization_level=3,
               search=True, tket=False, verify=False)
```

With `search=True` the driver builds these candidates, transpiles each with identical settings, and returns the one with the fewest two-qubit gates (ties broken by fewer ancillas, then fewer gates):

- the input circuit;
- Stage A only;
- full CTC;
- CTC without B3, without B1, and without A4;
- CTC with one ancilla reserved (when a budget is given).

Proposition 2 (never worse)**.** *With `search=True`, the returned circuit has at most as many two-qubit gates as `transpile(circuit)` with the same basis, optimisation level and seed.*

Proof. The input circuit is a candidate, transpiled with those same settings, and the minimum over a set is at most any of its members. □

Every candidate is transpiled with `transpile_exact` (§2), so no qubit permutation is hidden in layout metadata. The output keeps the caller's quantum and classical registers, in order. Added ancillas follow in an `AncillaRegister` named `ctc_anc`. The input circuit is never mutated. `verify(original, optimized)` checks equivalence by simulation:

- **up to 26 qubits:** a random state on all original qubits with ancillas in $|0\rangle$. The fidelity equals 1 only if the circuits agree *and* the ancillas return to $|0\rangle$.
- **27–28 qubits:** a mirror circuit S; opt; orig†; $S^\dagger$ with a random brickwork state preparation S.

Complexity. Let G be the number of gates:

- Lifting is $O(G)$, plus a small matrix check per controlled gate.
- Stage A sweeps are $O(GW)$ each, where W is the length of the commuting window scanned. The number of sweeps is bounded by the number of rewrites plus one.
- Stage B is $O(G \cdot L \cdot U)$ for L literals per cube and U look-ahead uses.

In practice the run time is dominated by the roughly six Qiskit transpilations of the search (§7.6).

7 Evaluation

Setup.

- Qiskit 2.5.2, qiskit-aer 0.17.2, pytket 2.18.3.
- Basis $\{\mathrm{CX}, R_z, \sqrt{X}, X\}$, optimisation level 3, `seed_transpiler=42`, `qubits_initially_zero=False` and no SWAP elision (§2) for every configuration.
- Two-qubit gates are counted after transpilation. Ancillas are the qubits added beyond the input.

Baselines:

- *Qiskit*: `transpile` of the input.
- *Clean per gate*: every MCX with ≥ 3 controls replaced by Maslov's construction on shared clean ancillas, then transpiled. This is the standard use of clean ancillas.
- *pytket*: Qiskit's output, then `FullPeepholeOptimise` with `allow_swaps=False`, mapped back to the original qubits by register name and index, then transpiled again. Its outputs are verified like CTC's.

Results were verified by simulation as follows:

- **every configuration up to 20 qubits**, except the curl-curl ablation rows;
- **the headline configurations up to 26 qubits**: the baselines, CTC and CTC without ancillas.

The curl-curl ablation rows were skipped to save time; they are built by the same code path as the verified configurations. In total 865 results passed, with 0 failures. 31 configurations were not simulated, being too wide or skipped as above.

7.1 Circuit families

Figure 3 and Table 2 separate three regimes.

- **Shared control (selector ladders, controlled register swaps).** This is the target case. For the ladder with $k = 8$: Qiskit 4 008, clean per gate 399, pytket 4 000, CTC 222 on 11 ancillas.
  - Against Qiskit the factor grows with k, because Qiskit's ancilla-free synthesis is superlinear in the number of controls.
  - Against per-gate clean ancillas it stays near 1.8–1.9×: the cache pays for each selector conjunction and register prefix once per ladder rather than once per gate.

  Controlled register swaps have no MCX gates, so the per-gate clean-ancilla baseline does not apply. CTC reduces them about 10× with only 2 ancillas. The 3-qubit selector is cached once, and every swap becomes a single-controlled Fredkin.
- **Multiplexed rotations.** A4 alone turns 2^m multi-controlled rotations into a 2^m-CX multiplexor, with no ancillas: a reduction of 8–24×.

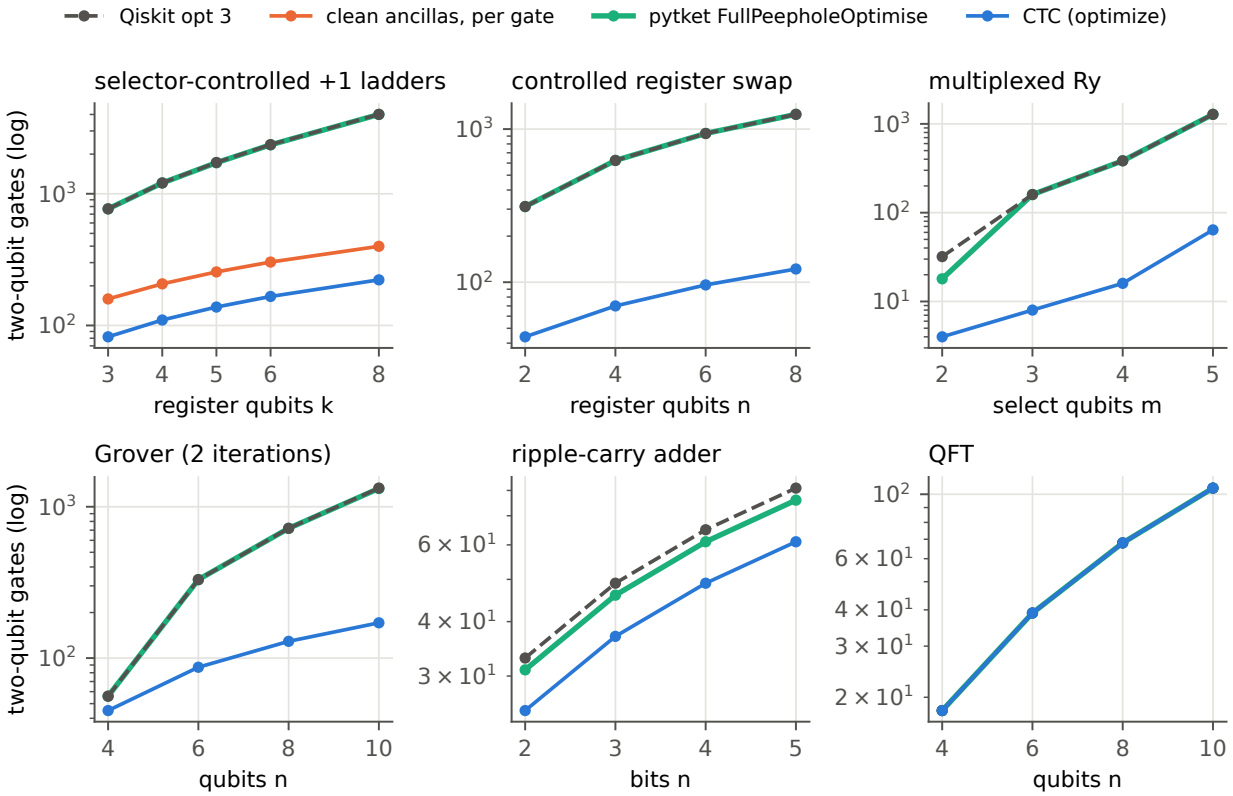


Figure 3: Two-qubit gates against size for six families. CTC is the default `optimize` call.

- **Oracle-style circuits (Grover).** The gain here does *not* come from sharing. Every oracle and diffuser MCZ is sandwiched between X or H layers that write all of its controls, so no cached conjunction survives from one to the next. B2 degenerates to synthesising each MCZ with clean ancillas, which is still far cheaper than Qiskit's ancilla-free synthesis (5.6× at $n = 8$).
- **Toffoli networks (ripple-carry adder).** Consecutive majority and unmajority blocks share Toffoli controls, and the cache reuses those ANDs, for a steady 1.3× on 2–5 ancillas.
- **No multi-controlled logic (QFT).** Controlled phases on two qubits give CTC nothing to rewrite. The search returns Qiskit's circuit unchanged (1.00×), as the never-worse guarantee requires.

7.2 What each step contributes

Tables 3–4 and Figure 4 answer two questions: what each step adds, and what is lost without it. The two views disagree, and the disagreement is informative.

- **Stage A is large on its own.** Without ancillas (columns +A1 to +A4 of Table 3), A1–A3 take the curl streaming blocks from 4 052 to 2 672 two-qubit gates. A3 and A4 take the relaxation block from 93 to 43.
- **B2 (the AND-cache) is the core.** Adding it gives the largest single drop on every circuit with multi-controlled logic, and removing it costs up to +1322%.
- **Most of Stage A is subsumed once B2 is on.** Leave-one-out, removing A1, A2 or A3 costs at most +6%, +2% and +4%. The reason is that the cache makes what those rewrites remove cheap anyway:
  - a redundant MCX pair under a cached selector costs 2 CX instead of $2(6c - 6)$;
  - a $ZXZX$ group costs 4 CX instead of 0.

  Stage A therefore matters most when ancillas are scarce (see the "CTC, 0 anc." columns and §7.5).
- **A4 is the exception.** The cache cannot synthesise a multiplexor, so removing A4 costs +950% on multiplexed rotations and +67% on the solver's relaxation step.

Table 2: Circuit families: two-qubit gates (ancillas in brackets). The last two columns give the CTC reduction over Qiskit and over the best of the three baselines.

family	size	qubits	Qiskit	clean/gate	pytket	**CTC**	CTC, 0 anc.	vs Qiskit	vs best
selector ladder	$k = 3$	7	768	159 (4)	768	**82** (6)	768	9.37×	1.94×
	$k = 4$	8	1 208	207 (5)	1 208	**110** (7)	1 208	10.98×	1.88×
	$k = 5$	9	1 728	255 (6)	1 720	**138** (8)	1 728	12.52×	1.85×
	$k = 6$	10	2 360	303 (7)	2 352	**166** (9)	2 360	14.22×	1.83×
	$k = 8$	12	4 008	399 (9)	4 000	**222** (11)	4 008	18.05×	1.80×
controlled swap	$n = 2$	7	312	—	312	**44** (2)	108	7.09×	7.09×
	$n = 4$	11	624	—	624	**70** (2)	216	8.91×	8.91×
	$n = 6$	15	936	—	936	**96** (2)	324	9.75×	9.75×
	$n = 8$	19	1 248	—	1 248	**122** (2)	432	10.23×	10.23×
multiplexed Ry	$m = 2$	3	32	—	18	**4** (0)	4	8.00×	4.50×
	$m = 3$	4	160	—	160	**8** (0)	8	20.00×	20.00×
	$m = 4$	5	384	—	384	**16** (0)	16	24.00×	24.00×
	$m = 5$	6	1 280	—	1 280	**64** (0)	64	20.00×	20.00×
Grover, 2 iter.	$n = 4$	4	56	—	56	**45** (1)	56	1.24×	1.24×
	$n = 6$	6	330	—	330	**87** (3)	330	3.79×	3.79×
	$n = 8$	8	720	—	720	**129** (5)	720	5.58×	5.58×
	$n = 10$	10	1 328	—	1 320	**171** (7)	1 328	7.77×	7.72×
ripple-carry adder	$n = 2$	6	33	—	31	**25** (2)	33	1.32×	1.24×
	$n = 3$	8	49	—	46	**37** (3)	49	1.32×	1.24×
	$n = 4$	10	65	—	61	**49** (4)	65	1.33×	1.24×
	$n = 5$	12	81	—	76	**61** (5)	81	1.33×	1.25×
QFT	$n = 4$	4	18	—	18	**18** (0)	18	1.00×	1.00×
	$n = 6$	6	39	—	39	**39** (0)	39	1.00×	1.00×
	$n = 8$	8	68	—	68	**68** (0)	68	1.00×	1.00×
	$n = 10$	10	105	—	105	**105** (0)	105	1.00×	1.00×

Table 3: Cumulative ablation: two-qubit gates as steps are added in the order A1, A2, A3, A4, B2, B1, B3 (`search=False`). The last column is the default `optimize` call. Up to A4 no ancillas are used; from B2 on, clean ancillas are added as needed.

circuit	Qiskit	+A1	+A2	+A3	+A4	+B2	+B1	+B3	**CTC**
selector ladder k=6	2 360	2 360	2 360	2 360	2 360	172	171	166	**166**
controlled swap n=6	936	936	936	936	936	97	97	96	**96**
multiplexed Ry m=4	384	384	384	384	16	16	16	16	**16**
Grover n=8	720	720	720	720	720	129	129	129	**129**
adder n=4	65	65	65	65	65	49	49	49	**49**
curl streaming N=8	4 052	3 452	2 820	2 672	2 672	334	343	324	**314**
curl π-rotation N=8	572	572	572	572	572	66	66	66	**66**
curl relaxation N=8	93	93	93	61	43	21	21	21	**21**
curl summations N=8	654	651	651	574	574	133	133	134	**133**
curl full timestep N=8	5 370	4 754	4 122	3 865	3 847	544	544	520	**520**

- **B3 (sibling flips) is a small, consistent refinement.** It is worth up to +6% where consecutive runs differ in one selector bit.
- **B1 (run ordering) never helped.** On the curl streaming blocks, removing it *lowers* the count by 3%. The source already groups gates by selector value, and reordering disturbs the control-prefix sharing that Qiskit's optimiser exploits. This is why `optimize` also tries a variant without B1: the default result on those blocks is lower than the all-steps configuration.

7.3 Random guarded circuits

The random circuits mix everything CTC models:

- open controls, controlled swaps and multiplexed rotations;
- CP, CRZ, CS, CY and CH gates, and uncontrolled H, R_z and $\sqrt{X}$;
- redundant pairs and $ZXZX$ groups.

CTC's median reduction is 3.48× (range 1.95–5.49×), against 1.05× for pytket and 1.03× for per-gate clean

Table 4: Leave-one-out ablation: the percentage of additional two-qubit gates when a single step is removed from the full set (0 = no effect).

circuit	all steps	−A1	−A2	−A3	−A4	−B1	−B2	−B3
selector ladder k=6	166	0	0	0	0	0	+1322%	+3%
controlled swap n=6	96	0	0	0	0	0	+875%	+1%
multiplexed Ry m=4	16	0	0	0	+950%	0	0	0
Grover n=8	129	0	0	0	0	0	+458%	0
adder n=4	49	0	0	0	0	0	+33%	0
curl streaming N=8	324	+2%	+1%	+1%	0	-3%	+725%	+6%
curl π-rotation N=8	66	0	0	0	0	0	+767%	0
curl relaxation N=8	21	0	0	0	+67%	0	+105%	0
curl summations N=8	134	0	0	+4%	0	0	+328%	-1%
curl full timestep N=8	520	+6%	+2%	+3%	+2%	0	+640%	+5%

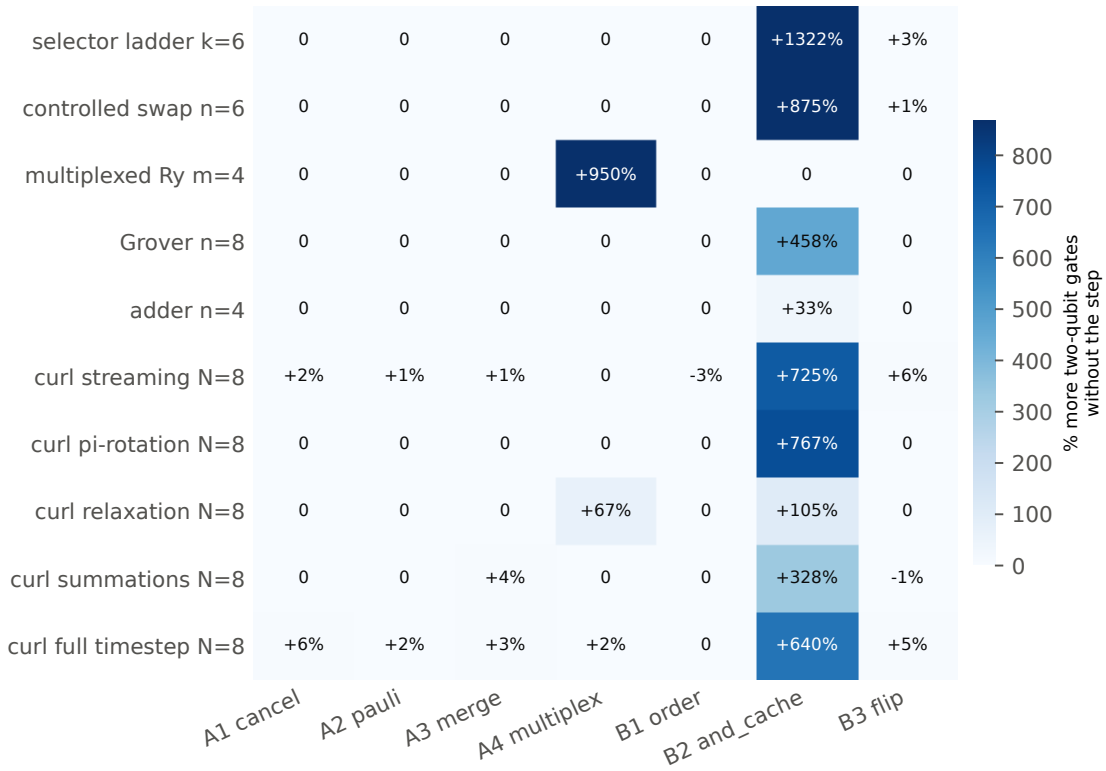


Figure 4: Leave-one-out step importance (the data of Table 4).

ancillas. CTC uses fewer two-qubit gates than pytket on 20 of 20 circuits. Without ancillas, Stage A and the driver's search still give a median of 3.09×.

7.4 A quantum PDE solver

CTC was developed on the circuits of a quantum solver for the magnetostatic curl-curl equation. Each timestep consists of:

- a controlled register rotation;
- three streaming blocks: 74 multi-controlled gates at $N = 8$, all sharing a 7-qubit copy/direction selector;
- a relaxation step of multiplexed rotations;
- a Hadamard-sandwiched summation block.

At $N = 8$ (Table 6), CTC reduces the full timestep from 2 461 CX (clean ancillas per gate) and 5 370 (Qiskit) to 520 on 9 ancillas. On the same 7 ancillas as the per-gate baseline it gives 530. The per-block ablation is in Table 3.

Table 5: 20 random guarded circuits (7 qubits, 40 random operations biased towards shared controls). Reduction factor = Qiskit two-qubit gates / method two-qubit gates.

method	median	min	max
Qiskit opt 3	1.00	1.00	1.00
clean ancillas, per gate	1.03	1.00	1.10
pytket FullPeepholeOptimise	1.05	1.00	1.24
CTC, no ancillas	3.09	1.72	4.98
CTC (`optimize`)	3.48	1.95	5.49

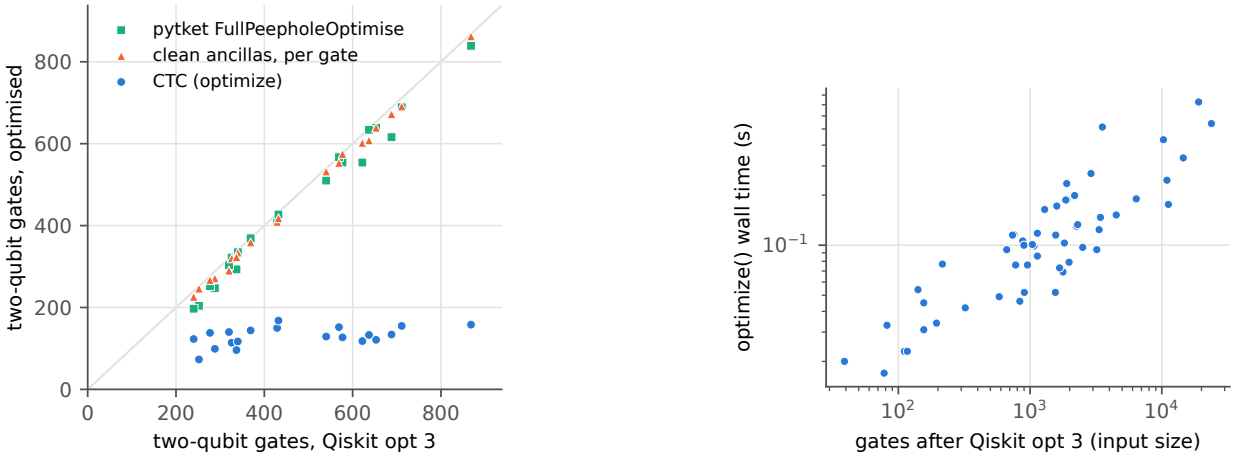


Figure 5: Left: random guarded circuits, optimised against Qiskit two-qubit gates (points below the diagonal are improvements). Right: wall time of `optimize` against input size over all benchmark circuits.

7.5 Ancilla trade-off

Ancillas are real qubits, so the budget is a design choice (Figure 6, Table 7). For the curl-curl timestep at $N = 8$ the count goes:

- 3 193 with no ancillas;
- 2 048 with one, where every residual MCX uses the Khattar–Gidney construction;
- 582 with six and 530 with seven.

Most of the gain arrives once the budget covers the depth of the shared selector chain, and further ancillas buy little. The curves are not always convex, because the cache is greedy. `ancilla_tradeoff` reports the best point found at each budget.

7.6 Run time

The median wall time of `optimize` over all benchmark circuits is 0.1 s. The slowest was 0.7 s, for curl_full_timestep (16). Time is dominated by transpiling the search candidates (Figure 5, right); `search=False` performs a single transpilation.

8 Related work

Multi-controlled gate synthesis.

- Barenco et al. [3] introduced the v-chain constructions with clean and dirty ancillas.
- Maslov [1] showed that relative-phase Toffolis halve the cost of the compute/uncompute rungs.
- Iten et al. [2] gave the $8c - 6$ dirty-ancilla construction.

Table 6: Full timestep of the quantum curl-curl solver at grid size N (two-qubit gates, ancillas in brackets). The last column is the reduction over per-gate clean ancillas, the best previous result for these circuits.

grid	qubits	Qiskit	clean/gate	pytket	**CTC**	CTC, 0 anc.	vs clean/gate
$N = 4$	16	3 790	2 009 (6)	3 774	**398** (8)	2 461	5.0×
$N = 8$	19	5 370	2 461 (7)	5 354	**520** (9)	3 193	4.7×
$N = 16$	22	7 046	2 929 (8)	7 030	**644** (11)	4 153	4.5×
$N = 32$	25	8 818	3 433 (9)	8 802	**768** (13)	5 209	4.5×

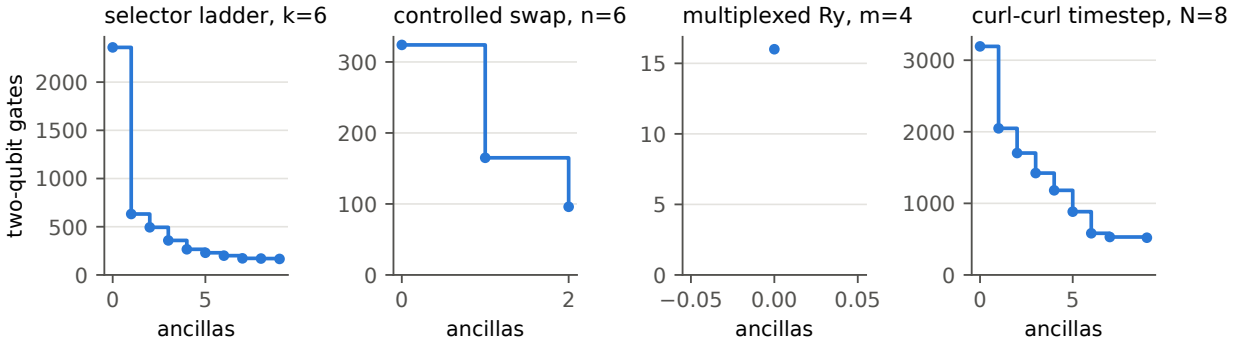


Figure 6: Best two-qubit count at each ancilla budget (`ancilla_tradeoff`).

- Khattar and Gidney [4] introduced conditionally clean ancillae, giving $6c - 6$ CX with a single ancilla.
- Gidney's temporary logical-AND [5] and Jones' measurement-based Toffoli [6] reduce uncomputation cost further when mid-circuit measurement is available. CTC is purely unitary.

All of these optimise one gate at a time. CTC shares the conjunctions across gates, and uses these constructions for what is left over.

Control flow and uncomputation. Yuan and Carbin [7] show that naive compilation of nested quantum control incurs polynomial overhead in T-complexity. They remove it by *conditional flattening*, which is closest in spirit to CTC's AND-cache. CTC works at the gate level on circuits whose control structure has already been flattened into gates, and targets two-qubit count. Unqomp and Reqomp [8, 9] synthesise uncomputation automatically for ancilla variables in a quantum programming language.

Rewriting and template methods.

- Template matching [10] and mixed-polarity Toffoli templates [11] include the merges A1 and A3 perform, in their reversible-logic setting.
- Multiplexor synthesis [12, 13] underlies A4.
- ZX-calculus optimisers [14, 15] and search- or learning-based optimisers [16, 17] are general-purpose and complementary.

CTC's output can be passed to them; pytket's peephole pass recovers little on it (§7).

Novelty, stated carefully. The individual identities are standard. What we believe is new is:

- combining them in one guarded-gate IR, with a sound commutation test;
- the look-ahead AND-cache, which extends the relative-phase argument across gate sequences under an ancilla budget;
- the numerically derived sibling-flip correction.

We have not performed an exhaustive literature search for the last two, and would welcome pointers to prior art.

Table 7: Two-qubit gates by ancilla budget (budget: count).

circuit	budget: two-qubit gates
selector ladder k=6	0: 2 360, 1: 632, 2: 494, 3: 358, 4: 266, 5: 230, 6: 200, 7: 172, 8: 170, 9: 166
controlled swap n=6	0: 324, 1: 165, 2: 96
multiplexed Ry m=4	0: 16
curl full timestep N=8	0: 3 193, 1: 2 048, 2: 1 703, 3: 1 423, 4: 1 182, 5: 884, 6: 582, 7: 530, 8: 530, 9: 520

9 Limitations

- **Two-qubit gates only.** T-count often falls as a side effect, since fewer multi-controlled rotations mean fewer arbitrary angles, but it is not optimised.
- **Ancillas.** Most of the gain needs clean ancillas. With no ancillas, CTC is limited to Stage A and the search.
- **Greedy.** The cache uses a fixed cost model and greedy extension, so it is not optimal and more ancillas can occasionally cost more. The search and `ancilla_tradeoff` mitigate this.
- **Scope of the IR.** Measurements, classical control and multi-qubit custom gates are opaque walls, and no rewrite crosses them. Barriers are removed.
- **Verification limits.** Exactness is proved per step and checked by simulation up to 26 qubits (28 by mirror). Wider outputs rely on the proofs and on verified outputs of the same code path at smaller sizes.

10 Conclusion

When a circuit is a program of quantum if-statements, its two-qubit cost is dominated by recomputing the same conditions. Treating multi-controlled gates as guarded operations makes four exact rewrites and a shared AND-cache possible. Together they give large, verified reductions on the circuits that have this structure, and a never-worse result on those that do not. The method is available as `optimize(circuit)`.

A Reproducing the results

```
pip install -e ctc_optimizer[all]
cd ctc_optimizer/benchmarks
python run_benchmarks.py --part worked      # and: families random curl tradeoff
python run_benchmarks.py --merge
python make_tables.py && python figures.py
cd ../paper && pdflatex CTC_PAPER.tex && pdflatex CTC_PAPER.tex
python ../tests/test_ctc.py                 # 12 tests incl. randomised soundness
```

All circuits (inputs and CTC outputs) are saved in `benchmarks/results/circuits/` as QPY, with SVG drawings for those of at most 300 operations. The curl-curl circuits require the `resource_estimation` project alongside the package.

References

[1] D. Maslov, *Advantages of using relative-phase Toffoli gates with an application to multiple control Toffoli optimization*, Phys. Rev. A **93**, 022311 (2016). arXiv:1508.03273.

[2] R. Iten, R. Colbeck, I. Kukuljan, J. Home, M. Christandl, *Quantum circuits for isometries*, Phys. Rev. A **93**, 032318 (2016). arXiv:1501.06911.

[3] A. Barenco et al., *Elementary gates for quantum computation*, Phys. Rev. A **52**, 3457 (1995). quant-ph/9503016.

[4] T. Khattar, C. Gidney, *Rise of conditionally clean ancillae for optimizing quantum circuits*, Quantum **9**, 1752 (2025). arXiv:2407.17966.

[5] C. Gidney, *Halving the cost of quantum addition*, Quantum **2**, 74 (2018). arXiv:1709.06648.

[6] C. Jones, *Novel constructions for the fault-tolerant Toffoli gate*, Phys. Rev. A **87**, 022328 (2013). arXiv:1212.5069.

[7] C. Yuan, M. Carbin, *The T-Complexity Costs of Error Correction for Control Flow in Quantum Computation*, Proc. ACM Program. Lang. **8** (PLDI 2024). arXiv:2311.12772.

[8] A. Paradis, B. Bichsel, S. Steffen, M. Vechev, *Unqomp: synthesizing uncomputation in quantum circuits*, PLDI 2021. doi:10.1145/3453483.3454040.

[9] A. Paradis, B. Bichsel, M. Vechev, *Reqomp: space-constrained uncomputation for quantum circuits*, Quantum (2024). arXiv:2212.10395.

[10] D. Maslov, G. W. Dueck, D. M. Miller, *Toffoli network synthesis with templates*, IEEE TCAD **24**(6), 807–817 (2005).

[11] M. M. Rahman, J. E. Rice, *Templates for positive and negative control Toffoli networks*, Reversible Computation 2014, LNCS 8507, 125–136.

[12] M. Möttönen, J. J. Vartiainen, V. Bergholm, M. M. Salomaa, *Quantum circuits for general multiqubit gates*, Phys. Rev. Lett. **93**, 130502 (2004). quant-ph/0404089.

[13] V. V. Shende, S. S. Bullock, I. L. Markov, *Synthesis of quantum logic circuits*, IEEE TCAD **25**(6) (2006). quant-ph/0406176.

[14] A. Kissinger, J. van de Wetering, *PyZX: large scale automated diagrammatic reasoning*, EPTCS **318** (2020). arXiv:1904.04735.

[15] K. Staudacher, T. Guggemos, S. Grundner-Culemann, W. Gehrke, *Reducing 2-qubit gate count for ZX-calculus based quantum circuit optimization*, EPTCS **394** (2023). arXiv:2311.08881.

[16] M. Xu et al., *Quartz: superoptimization of quantum circuits*, PLDI 2022. arXiv:2204.09033.

[17] A. Xu, A. Molavi, S. Tannu, A. Albarghouthi, *Optimizing quantum circuits, fast and slow*, ASPLOS 2025. arXiv:2411.04104.